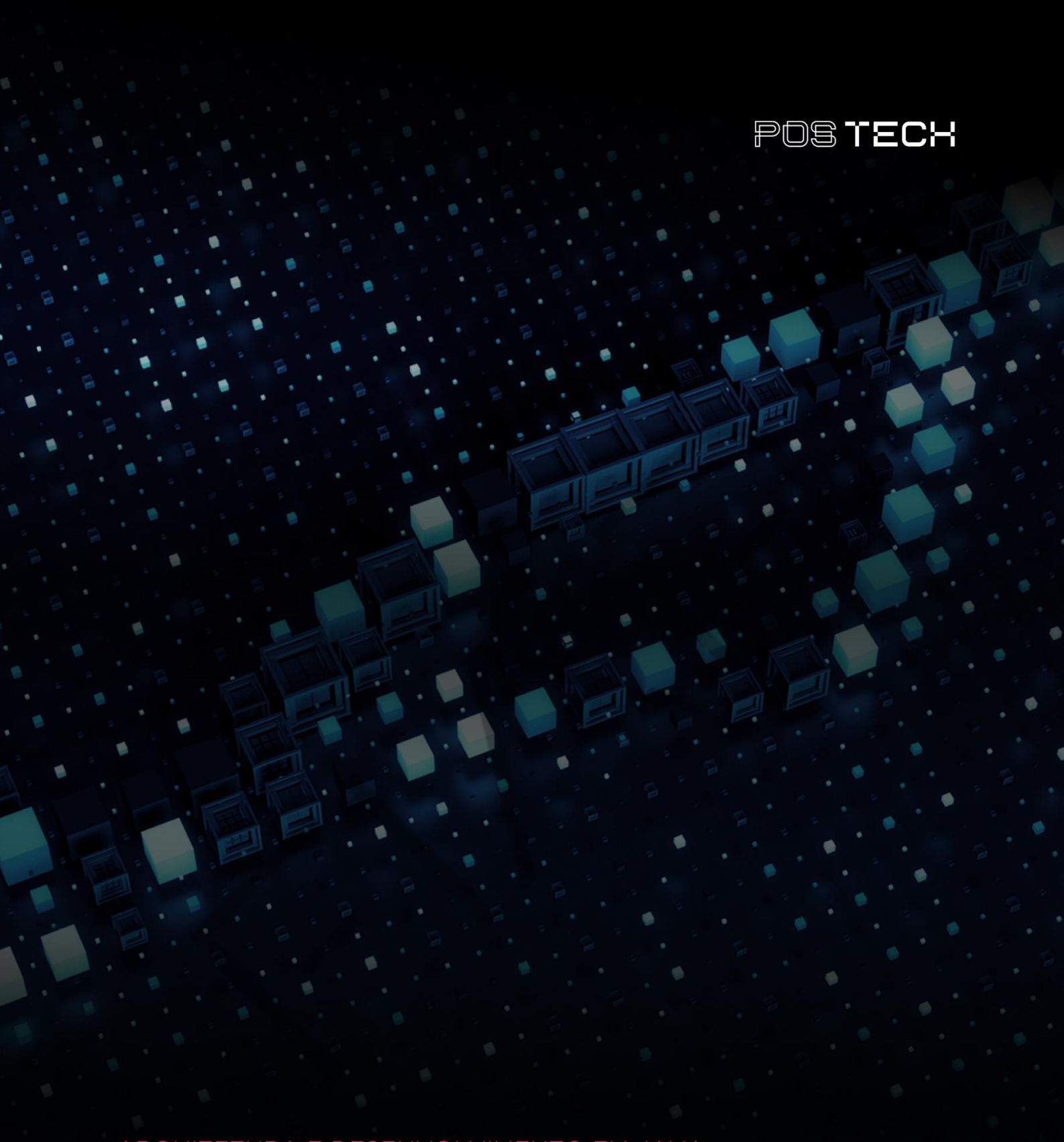


PDS TECH

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 3 | AULA 02 -

CONSTRUINDO APIS COM GRPC

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3

HANDS ON 4

SAIBA MAIS 5

O QUE VOCÊ VIU NESTA AULA? 10

REFERÊNCIAS 11

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Nesta aula, veremos como construir uma API completa utilizando gRPC. A proposta é colocar em prática os conceitos introduzidos anteriormente, transformando teoria em uma aplicação, enquanto exploramos as features do gRPC para criar APIs com alta eficiência.

Iniciaremos pela definição de um serviço utilizando ProtoBuf, onde estruturaremos os métodos e mensagens que compõem a API. Em seguida, trabalharemos com a geração automática de código, um recurso essencial do gRPC, que facilita a implementação tanto no lado do servidor quanto do cliente. No servidor, desenvolveremos a lógica necessária para atender às requisições e, no cliente, validaremos a comunicação e o funcionamento do serviço.

Além disso, vamos abordar o entendimento dos diferentes tipos de comunicação suportados pelo gRPC (Unary, Server Streaming, Client Streaming e Bidirectional Streaming) e discutiremos como aplicá-los estrategicamente em diferentes cenários, transformando conceitos em soluções concretas.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Nesta aula, vamos aplicar os conceitos discutidos anteriormente para implementar uma solução utilizando gRPC. O processo será estruturado e detalhado, abrangendo desde a configuração inicial do ambiente até a implementação e teste dos serviços.

Começaremos pela configuração do ambiente de desenvolvimento, garantindo que todas as dependências e ferramentas necessárias estejam corretamente integradas. Em seguida, definiremos os serviços utilizando uma linguagem de descrição que especifica os métodos e mensagens a serem utilizados. Essa definição será fundamental para estabelecer os contratos de comunicação.

Utilizaremos ferramentas específicas para gerar o código necessário a partir da definição do serviço, criando as bases para a implementação. Em seguida, implementaremos a lógica dos serviços, configuraremos o servidor para responder às requisições e prepararemos um cliente que será responsável por testar a comunicação.

Durante a implementação, exploraremos os diferentes padrões de comunicação suportados pelo gRPC. Para cada padrão, analisaremos sua aplicabilidade e implementamos exemplos práticos, demonstrando como atender a diferentes demandas de sistemas distribuídos.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

Vamos entender melhor a teoria por trás do padrão de comunicação Unary. A comunicação do tipo Unary é a forma mais direta de interação no gRPC. Nesse modelo, o cliente realiza uma única requisição ao servidor, que, por sua vez, retorna uma única resposta. Esse padrão é frequentemente utilizado em operações onde uma chamada pontual é suficiente para atender à necessidade de troca de informações.

O fluxo de funcionamento do Unary é simples e segue etapas bem definidas. Primeiramente, o cliente envia uma requisição ao servidor contendo os dados necessários para a execução da operação. Essa requisição é composta por uma mensagem definida no arquivo ProtoBuf, que estabelece de forma clara e precisa os campos que devem ser enviados.

Ao receber a requisição, o servidor processa os dados conforme a lógica implementada no método correspondente ao serviço. Esse processamento pode envolver cálculos, consultas a bases de dados ou outras operações necessárias para gerar a resposta esperada. Após o término do processamento, o servidor envia de volta uma única mensagem de resposta ao cliente, finalizando o ciclo de comunicação.

O tipo Unary é amplamente utilizado em cenários que não requerem trocas contínuas de dados ou interações complexas. Exemplos comuns incluem validações simples, consultas a serviços externos, cálculos rápidos ou operações CRUD (criação, leitura, atualização e exclusão). Sua eficiência e simplicidade tornam o padrão adequado para operações que precisam de baixa latência e mínimo overhead na comunicação.

Apesar de sua simplicidade, o padrão Unary ainda mantém as vantagens do gRPC, como alto desempenho, suporte a múltiplas linguagens e uso do ProtoBuf para serialização compacta de dados. Além disso, ele é totalmente compatível com outras funcionalidades do gRPC, como interceptores e autenticação, permitindo que seja integrado a sistemas mais robustos.

O padrão Unary é um excelente ponto de partida para compreender a dinâmica do gRPC. Mesmo não utilizando o gRPC, muitas vezes ainda utilizamos a comunicação unary. Esse padrão também estabelece a base necessária para explorar

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

padrões de comunicação mais avançados, como Streaming, e demonstra como a tecnologia pode ser utilizada de forma eficiente em operações comuns do dia a dia.

Outro padrão, o Server Streaming, no gRPC, é utilizado em cenários onde o cliente realiza uma única requisição ao servidor, mas o servidor retorna uma sequência de mensagens. Essa abordagem é ideal para situações em que uma operação resulta em múltiplos dados que precisam ser transmitidos gradativamente ao cliente.

O fluxo de funcionamento do Server Streaming segue uma estrutura bem definida. Inicialmente, o cliente envia uma requisição ao servidor com os parâmetros necessários para a operação, encapsulados em uma mensagem ProtoBuf. O servidor, ao receber essa requisição, inicia o processamento e começa a enviar uma série de mensagens de resposta, uma por vez, até que todos os dados estejam transmitidos. Quando o servidor conclui o envio de todas as mensagens, ele finaliza a transmissão, encerrando o fluxo de dados.

Esse padrão é particularmente útil em aplicações que precisam entregar grandes volumes de dados ou onde a entrega gradual é mais eficiente do que o envio de tudo em uma única resposta. Exemplos típicos incluem a transmissão de logs, listas de itens ou dados em tempo real, como atualizações de mercado financeiro ou notificações de eventos.

No Server Streaming, cada mensagem enviada pelo servidor é processada individualmente pelo cliente à medida que chega. Isso permite ao cliente consumir os dados de forma incremental, otimizando o uso de memória e reduzindo o tempo de espera para começar a processar as informações.

Embora o Server Streaming ofereça benefícios significativos em termos de desempenho e flexibilidade, ele também requer atenção especial na implementação. É necessário garantir que o servidor administre corretamente o fluxo de dados, evitando sobrecarga no cliente. Além disso, a comunicação baseada em streams exige que erros sejam tratados de maneira robusta, pois problemas podem ocorrer durante qualquer etapa do envio das mensagens.

Esse padrão é uma escolha estratégica para cenários onde a entrega gradual de informações agrega valor e sua correta implementação pode melhorar significativamente a experiência do cliente e o desempenho do sistema todo.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Também temos o padrão de comunicação Client Streaming no gRPC. Ele é utilizado em cenários onde o cliente envia uma sequência de mensagens ao servidor como parte de uma única requisição e o servidor responde com uma única mensagem após processar todas as mensagens recebidas (o inverso do padrão Server Streaming). Esse modelo é especialmente útil quando o cliente precisa transmitir um conjunto de dados de maneira incremental, sem sobrecarregar o servidor com uma única grande requisição.

O funcionamento do Client Streaming segue um fluxo estruturado. O cliente inicia a comunicação estabelecendo o canal e começa a enviar as mensagens de forma contínua. Cada mensagem é transmitida de maneira independente, utilizando o stream de comunicação estabelecido. O servidor, por sua vez, recebe essas mensagens em tempo real, mas só inicia o processamento completo após o cliente indicar que todas as mensagens foram enviadas e o stream está encerrado. Com todas as mensagens recebidas, o servidor processa os dados e envia uma única resposta consolidada ao cliente, encerrando a comunicação.

Esse padrão é amplamente utilizado em situações onde os dados precisam ser enviados ao servidor de forma gradual ou parcelada. Exemplos incluem o envio de arquivos em partes, upload de grandes volumes de informações ou coleta de dados de sensores em tempo real, onde o servidor processa o conjunto final de informações para fornecer um resultado.

A comunicação Client Streaming oferece vantagens significativas, como a possibilidade de enviar grandes quantidades de dados sem sobrecarregar a memória, já que o envio é feito em partes. No entanto, também introduz desafios específicos. O cliente precisa gerenciar adequadamente o envio das mensagens e sinalizar corretamente o término do stream. O servidor, por sua vez, deve garantir que todas as mensagens sejam processadas de maneira eficiente, mesmo em casos de falhas ou interrupções no fluxo.

Além disso, é importante implementar estratégias robustas de tratamento de erros para lidar com situações como falhas na conexão ou inconsistências nos dados recebidos. Essas medidas são essenciais para garantir a confiabilidade e a consistência do sistema.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O padrão Client Streaming destaca a flexibilidade do gRPC para atender a cenários complexos de transmissão de dados, permitindo que o cliente controle o ritmo e o volume das mensagens enviadas. Sua aplicação adequada pode otimizar o desempenho e a escalabilidade de sistemas distribuídos que demandam transmissão contínua de informações.

Para finalizar os quatro modelos de comunicação dentro do gRPC, temos a comunicação Bidirectional Streaming. É o padrão mais avançado e flexível, permitindo que cliente e servidor troquem mensagens simultaneamente e de forma independente por meio de um stream bidirecional. Nesse modelo, ambos os lados podem enviar e receber mensagens em tempo real, sem a necessidade de esperar que o outro finalize sua parte no processo. Isso torna o Bidirectional Streaming ideal para cenários que demandam interações contínuas e dinâmicas.

O fluxo desse padrão inicia quando o cliente estabelece a conexão e envia a primeira mensagem ao servidor. Assim que o stream é iniciado, tanto o cliente quanto o servidor podem transmitir mensagens a qualquer momento, de acordo com a lógica da aplicação. O stream permanece ativo enquanto os lados continuarem enviando mensagens e só será encerrado quando cliente e servidor concluírem suas operações e explicitamente finalizarem o fluxo.

O Bidirectional Streaming é amplamente utilizado em aplicações que requerem comunicação contínua ou bidirecional em tempo real. Exemplos comuns incluem chats interativos, sistemas de monitoramento onde sensores enviam dados continuamente e o servidor responde com instruções e processamento de streams de dados financeiros, onde eventos são enviados e respostas são calculadas dinamicamente.

Esse padrão oferece vantagens significativas, como a redução da latência, já que cliente e servidor não precisam esperar que o outro finalize sua comunicação para iniciar suas próprias operações. Além disso, ele proporciona maior eficiência para interações complexas, permitindo que dados sejam trocados de maneira fluida e em paralelo.

No entanto, a implementação do Bidirectional Streaming exige cuidado especial. O gerenciamento do fluxo de mensagens é mais complexo, já que o cliente e o servidor operam de forma assíncrona. É fundamental implementar controle de

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

fluxo, para evitar sobrecargas de dados, e estratégias robustas de tratamento de erros, pois falhas em qualquer um dos lados podem comprometer a comunicação.

Outro ponto a considerar é o impacto na lógica da aplicação, que precisa ser projetada para lidar com mensagens que chegam de forma não sequencial ou inesperada. Isso requer um entendimento profundo dos requisitos da aplicação e do comportamento esperado do fluxo de mensagens.

O padrão Bidirectional Streaming destaca toda a potência e flexibilidade do gRPC para atender a cenários modernos e altamente interativos. Quando bem implementado, ele permite a criação de sistemas distribuídos dinâmicos, escaláveis e que oferecem uma experiência de comunicação fluida e eficiente.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, avançamos na prática com a implementação de APIs utilizando o gRPC, consolidando os conceitos discutidos anteriormente. Exploramos como o gRPC se destaca como uma solução moderna para comunicação eficiente entre sistemas distribuídos, utilizando o ProtoBuf como mecanismo padrão para serialização de dados e garantindo alto desempenho e compactação das mensagens.

Com as práticas abordadas, você não apenas implementou uma API funcional com gRPC, mas também desenvolveu uma compreensão mais profunda sobre como essa tecnologia pode ser utilizada para atender a diferentes demandas de sistemas distribuídos. A partir daqui, você está preparado e preparada para aplicar esses conceitos em projetos mais complexos e explorar o potencial completo do gRPC em ambientes reais.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

GRPC.IO**. Core concepts, architecture and lifecycle**. 2024. Disponível em: <https://grpc.io/docs/what-is-grpc/core-concepts/>. Acesso em: 09 dez. 2024.

GRPC.IO. **Server Streaming RpRPCc**. 2024. Disponível em: <https://grpc.io/docs/what-is-grpc/core-concepts/#server-streaming-rpc>. Acesso em: 09 dez. 2024.

GRPC.IO**. Bidirectional Streaming RPC**. 2024. Disponível em: <https://grpc.io/docs/what-is-grpc/core-concepts/#bidirectional-streaming-rpc>. Acesso em: 09 dez. 2024.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

Remote Procedure Call. gRPC. Protocol Buffers. Serialização. Deserialização. Mensagens Escaláveis. Interoperabilidade. Compactação de Dados. Microsserviços. HTTP/2. Streaming. Eficiência em Redes. Imutabilidade. Contratos Protobuf. Well-Known Types. Versionamento de Mensagens. Boas Práticas. Integração Cliente-Servidor. Cloud-Native. CNCF. Automação de Código.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com